



**Escuela Superior
de Ingeniería y Tecnología**
Universidad de La Laguna

Trabajo de Fin de Grado

Grado en Ingeniería Informática

Título del trabajo

Design and implementation of a modular IoT device

Alejandro Rodríguez Mederos

La Laguna, 25 de mayo de 2026

D.**Ivan Castilla Rodríguez**, profesor Titular de Universidad adscrito al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, como tutor

D.**Santiago Torres Álvarez**, profesor Titular de Universidad adscrito al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, como cotutor.

C E R T I F I C A (N)

Que la presente memoria titulada:

“Diseño e implementación de un dispositivo IoT modular”

ha sido realizada bajo su dirección por D./Dña **Nombre Apellido1 Apellido2**.

Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos, firman la presente en La Laguna a 25 de mayo de 2026

Agradecimientos

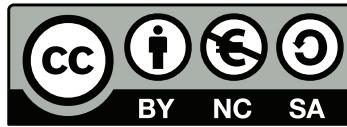
En primer lugar, me gustaría expresar mi más sincero agradecimiento a mis tutores del Trabajo de Fin de Grado, D. Iván Castilla Rodríguez y D. Santiago Torres Álvarez, ya que, sin su apoyo, orientación y dedicación, este proyecto no habría sido posible.

Desde el momento en que se presentó la idea inicial del proyecto, ambos mostraron un gran interés en el trabajo, proporcionando orientación constante y ayudándome a enfocar su desarrollo hacia una dirección realista, viable y técnicamente adecuada. Asimismo, agradezco profundamente su disponibilidad, implicación y seguimiento continuo durante todas las fases del proyecto, así como sus consejos y aportaciones para mejorar tanto el diseño como el planteamiento general del sistema.

También quisiera agradecer a David Abreu el tiempo dedicado a explicarme distintos conceptos relacionados con los sistemas de seguridad aplicados a entornos embebidos e IoT, aportando una visión adicional que resultó de gran utilidad durante el desarrollo del proyecto.

Finalmente, agradezco a todas aquellas personas que, de una forma u otra, han contribuido y apoyado la realización de este Trabajo de Fin de Grado.

Licencia



© Esta obra está bajo una licencia de Creative Commons
Reconocimiento-NoComercial-CompartirIgual 4.0
Internacional.

Bajo esta licencia, se permite compartir y adaptar el material siempre que se reconozca la autoría, no se haga un uso comercial del mismo y las adaptaciones se distribuyan bajo la misma licencia que el original.

Resumen

El presente Trabajo de Fin de Grado tiene como objetivo el diseño e implementación de un dispositivo IoT modular basado en el microcontrolador ESP32-S3. El sistema propuesto permite la integración de distintos módulos hardware mediante buses de comunicación estándar, ofreciendo una plataforma flexible y reutilizable orientada a la monitorización, automatización y control de procesos.

Como contribución principal, se ha desarrollado un lenguaje específico de dominio (DSL), diseñado para simplificar la interacción con el hardware sin necesidad de recompilar el firmware. El lenguaje permite definir dispositivos, controlar el flujo de ejecución, gestionar variables, comunicarse con periféricos I2C y registrar eventos mediante archivos de log almacenados en tarjeta microSD.

Complementariamente, el dispositivo incorpora un servidor HTTP embebido que sirve una interfaz web accesible desde cualquier navegador, permitiendo gestionar archivos, editar programas en el DSL, consultar documentación integrada y supervisar el estado del sistema.

El diseño software sigue una arquitectura modular en capas, facilitando el mantenimiento y la escalabilidad del sistema. Se ha realizado además un análisis de consumo energético considerando distintos escenarios de operación, y se ha elaborado un diseño conceptual del dispositivo físico que sirve de base para futuras iteraciones del prototipo.

Palabras clave: IoT, ESP32-S3, DSL, lenguaje específico de dominio, sistemas embebidos, modular, I2C, automatización, prototipado.

Abstract

This Final Degree Project aims to design and implement a modular IoT device based on the ESP32-S3 microcontroller. The proposed system allows the integration of different hardware modules through standard communication buses, offering a flexible and reusable platform oriented towards monitoring, automation and process control.

As the main contribution, a domain-specific language (DSL) has been developed, designed to simplify interaction with the hardware without requiring firmware recompilation. The language allows users to define devices, control execution flow, manage variables, communicate with I2C peripherals and log events to files stored on a microSD card.

Additionally, the device incorporates an embedded HTTP server that serves a web interface accessible from any browser, enabling file management, DSL program editing, access to integrated documentation and system status monitoring.

The software design follows a modular, layered architecture that facilitates maintainability and scalability. An energy consumption analysis has been carried out considering different operating scenarios, and a conceptual physical design of the device has been produced as a foundation for future prototype iterations.

Keywords: IoT, ESP32-S3, DSL, domain-specific language, embedded systems, modular, I2C, automation, rapid prototyping.

Índice general

Capítulo 1. Introducción	1
1.1. Contexto y motivación	1
1.2. Estructura del documento	2
Capítulo 2. Arquitectura del dispositivo	4
2.1. Hardware	4
2.2. Software	5
2.3. Casos de uso	6
Capítulo 3. Puesta en marcha	7
3.1. Conectividad (Network)	9
3.1.1. Modo Access Point (AP)	9
3.1.2. Modo Station (STA)	10
3.1.3. Gestión de conectividad y seguridad	10
3.2. Servidor (Server)	11
3.3. Interpreter	14
3.4. Expansión (Expansion)	15
3.5. Display	16
3.6. Tasks (Gestión de tareas)	18
Capítulo 4. Gestión energética y sistema de alimentación	21
4.1. Sistemas de baterías	21
4.2. Sistema de carga	22
4.3. Estimación de consumo y autonomía	22
4.4. Sistema de refrigeración y control térmico	23
Capítulo 5. Módulos conectados al dispositivo	24
5.0.1. Sistema de interacción con el usuario	24
5.1. Sistema de botones	24
5.1.1. Sistema de señalización LED	25
5.2. Módulo de expansión hardware	25
5.3. Sensores utilizados en el prototipo	25
Capítulo 6. Diseño final	27
6.1. Distribución física del dispositivo	28
6.2. Coherencia con la arquitectura del sistema	29
Capítulo 7. Resumen y Conclusiones / Summary and Conclusions	30
7.1. Resumen	30
7.2. Conclusiones	31
7.3. Summary	31

7.4. Conclusions	32
7.5. Trabajo Futuro	32
7.6. Future Work	33
7.7. Uso de la Inteligencia Artificial	34
7.8. Use of Artificial Intelligence	34
Capítulo 8. Presupuesto	36
8.1. Coste de mano de componentes	36
8.2. Coste de mano de obra	37
8.3. Coste total del proyecto	37
Capítulo 9. Bibliografía	38
Apéndice A. Componentes del bloque Components	39
A.1. IO	39
A.2. Lexer	39
A.3. Locator	40
A.4. Logger	40
A.5. Observer	40
Apéndice B. Especificación del lenguaje DSL	41
B.0.1. Estructura general del lenguaje	41
B.0.2. Ejemplo de inicio rápido	41
B.0.3. Definición de dispositivos	42
B.0.4. Escritura en dispositivos	42
B.0.5. Variables y operaciones	43
B.0.6. Control de flujo	43
B.0.7. Sistema de logs e impresión	44
B.0.8. Comunicación I2C	44
B.0.9. Expansor de pines (PCF8574)	45
B.0.10. Utilidades	45
B.0.11. Ejemplo completo	46
B.0.12. Referencia rápida	46

Índice de figuras

3.1. Representación conceptual del diseño del software.	7
3.2. Representación conceptual del brain.	8
3.3. Interfaz remota del configuración de la conectividad.	11
3.4. Interfaz remota del dashboard.	13
3.5. Representación conceptual de las conexiones.	16
3.6. Representación conceptual de las tasks.	19
6.1. Diseño exterior del dispositivo.	27
6.2. Distribución interna de los componentes.	28

Índice de tablas

4.1. Consumo estimado de los componentes del sistema	22
8.1. Coste de los componentes principales del sistema	36
8.2. Estimación del coste de desarrollo	37
8.3. Estimación del coste de desarrollo	37

Capítulo 1

Introducción

1.1. Contexto y motivación

En la actualidad, los sistemas basados en el Internet of Things (IoT) han experimentado un notable crecimiento, dando lugar a una amplia variedad de dispositivos diseñados para cubrir necesidades específicas en ámbitos como la automatización, la monitorización y el control de procesos. Sin embargo, muchas de estas soluciones están altamente especializadas, lo que implica la necesidad de utilizar diferentes dispositivos para cada aplicación concreta.

En este ámbito pueden encontrarse distintas soluciones, cada una enfocada en una necesidad específica, como por ejemplo la monitorización en tiempo real del suelo y de las condiciones meteorológicas. También existen soluciones que permiten adaptarse a las necesidades del usuario, aunque dependen de servicios externos o requieren amplios conocimientos sobre el área en la que se trabaja, como ocurre con algunos dispositivos IoT capaces de capturar datos mediante llamadas a APIs externas. Todos estos ejemplos tienen un punto en común: son dispositivos diseñados para resolver necesidades concretas. Algunos permiten adaptar su funcionamiento mediante modificaciones en el software; no obstante, esto requiere conocimientos técnicos y, en muchos casos, una investigación adicional por parte del usuario.

Esta situación genera una dependencia de soluciones cerradas y poco flexibles, dificultando la reutilización del hardware y aumentando tanto el coste como la complejidad de implementación de nuevos sistemas.

A esto se suma la necesidad de investigar exhaustivamente cuál de los diferentes dispositivos disponibles se adapta mejor a las necesidades o problemas que se desean resolver. En muchos casos, incluso es necesario adquirir los dispositivos para comprobar si realmente cumplen con los requisitos planteados. Además, si surge la necesidad de adaptar el hardware, podría comprometerse la integridad del dispositivo.

Es aquí donde surge el presente Trabajo de Fin de Grado, con la finalidad de abordar todos estos problemas mediante el desarrollo de un dispositivo capaz de adaptarse a las necesidades del usuario, y no al contrario. Se busca evitar que el usuario tenga que profundizar en el funcionamiento de dispositivos que quizá no se ajusten a sus necesidades, ahorrándole tiempo tanto en el proceso de prueba de hardware como en la comprensión del software asociado.

Por ello, se propone un dispositivo capaz de reducir los tiempos de investigación, aumentar la capacidad del usuario para iterar sobre una misma plataforma en busca de soluciones, reutilizar el dispositivo en diferentes ámbitos e incluso expandir su funcionalidad dentro de un mismo contexto. Además, se pretende que el usuario pueda utilizar el dispositivo como un centro de operaciones desde el que seleccionar y ejecutar distintas herramientas sin necesidad de desarrollar software complejo

para cada caso concreto. Todo ello con la finalidad de abstraer al usuario final tanto del software como del hardware. En definitiva, se plantea un dispositivo totalmente flexible y preparado para adaptarse a cualquier tipo de necesidad.

Asimismo, el dispositivo se plantea como una solución portátil, ya que debe estar orientado a diferentes perfiles de usuario y adaptarse a las necesidades particulares de cada uno de ellos, esto engloba a perfiles que quieran utilizar para monitorizar sus actividades al aire libre o en deportes con condiciones más complejas.

La idea principal es que tanto usuarios avanzados como usuarios con menos conocimientos técnicos puedan utilizar el dispositivo de forma rápida y sencilla, permitiendo además que aquellos que lo deseen puedan profundizar en su funcionamiento interno y personalizarlo según sus necesidades.

Para llevar a cabo este dispositivo, será necesario tener en cuenta los siguientes objetivos:

- Diseñar un dispositivo base reutilizable para múltiples aplicaciones.
- Permitir la ampliación del dispositivo de manera rápida y sencilla.
- Facilitar la comunicación con el dispositivo sin necesidad de herramientas externas.
- Facilitar la creación, gestión y automatización de herramientas.
- Reducir el tiempo y los recursos necesarios para desarrollar soluciones específicas.

1.2. Estructura del documento

El presente documento se organiza en los siguientes capítulos:

- Capítulo 1: Introducción Se expone el contexto general del proyecto, la motivación que lo justifica, los objetivos planteados y los casos de uso considerados para el sistema desarrollado.
- Capítulo 2: Arquitectura del dispositivo Se describe la arquitectura general del sistema, tanto a nivel hardware como software, incluyendo las principales decisiones de diseño y los componentes que lo forman.
- Capítulo 3: Puesta en marcha Se detalla la arquitectura software del sistema, así como la organización por capas y el funcionamiento interno de los principales módulos.
- Capítulo 4: Gestión energética y sistema de alimentación Se analiza el sistema de alimentación del dispositivo, el consumo energético estimado y las soluciones planteadas para garantizar la autonomía del sistema.
- Capítulo 5: Módulos conectados al dispositivo Se describen los periféricos y módulos hardware utilizados, así como su integración dentro de la arquitectura del sistema.
- Capítulo 6: Diseño final Se presenta el diseño físico del dispositivo y la distribución de sus componentes, incluyendo aspectos de usabilidad y refrigeración.
- Capítulo 7: Conclusiones y trabajo futuro Se recogen las conclusiones del proyecto, el grado de cumplimiento de los objetivos y las posibles líneas de mejora futura.
- Capítulo 8: Presupuesto Se detalla el coste económico del proyecto, incluyendo componentes hardware y estimación de mano de obra.

- Capítulo 9: Bibliografía Se incluyen las referencias bibliográficas y fuentes utilizadas durante el desarrollo del proyecto.

Capítulo 2

Arquitectura del dispositivo

2.1. Hardware

Con la finalidad de gestionar las necesidades anteriormente mencionadas, será necesario contar con un hardware lo suficientemente potente como para procesar toda la información que el usuario pueda requerir y, al mismo tiempo, proporcionarle las herramientas necesarias para interactuar con el dispositivo de manera cómoda y eficiente. Por este motivo, el núcleo central del sistema estará basado en un microcontrolador ESP32-S3.

Este microcontrolador ofrece altas prestaciones, entre las que destaca su conectividad Wi-Fi, característica que desempeñará un papel fundamental en el desarrollo del dispositivo y que será tratada en profundidad más adelante. Además, dispone de una arquitectura de doble núcleo, lo que permite separar responsabilidades entre diferentes tareas del sistema, evitando bloqueos completos y mejorando la estabilidad y el rendimiento general del dispositivo.

En este sentido, existen otras alternativas en el mercado, como el Arduino Uno R4, que también incorpora capacidades de conectividad similares. Sin embargo, dichas funcionalidades son posibles gracias a la integración de un ESP32 como coprocesador, lo que reafirma la relevancia de esta familia de microcontroladores para el desarrollo de soluciones IoT avanzadas.

A esto se suma la posibilidad de establecer comunicaciones directas entre dispositivos basados en ESP32 mediante mecanismos específicos como ESP-NOW, lo que permite la creación de redes de baja latencia y comunicación directa entre nodos sin necesidad de infraestructura de red convencional. Esta característica amplía las posibilidades del sistema, ya que habilita una capa adicional de conectividad entre dispositivos cuando el caso de uso lo requiera.

Respecto al resto de componentes hardware, cabe destacar la incorporación de una pantalla que, idealmente, será táctil. Esto permitirá reducir la complejidad de interacción y adaptar el dispositivo a interfaces con las que el usuario ya está familiarizado, como ocurre en el caso de los teléfonos inteligentes. Esta capa de interacción permitirá al usuario utilizar de forma directa las diferentes herramientas y aplicaciones desarrolladas para el dispositivo.

Este aspecto resulta especialmente importante, ya que posibilita una comunicación directa entre el usuario y el sistema sin necesidad de recurrir a otros dispositivos electrónicos externos, facilitando así su uso en cualquier entorno y situación.

Otro de los componentes clave será el encargado de abstraer la integración del hardware externo que el usuario desee añadir al sistema. El objetivo es permitir la conexión y desconexión de distintos módulos de manera sencilla, evitando procesos complejos de cableado y reduciendo el riesgo de comprometer la integridad del dispositivo. Para ello, este componente deberá mantener una comunicación estable

con el núcleo principal del sistema, habilitando interfaces que permitan acoplar diferentes módulos de expansión de forma segura y transparente para el usuario.

El dispositivo contará con un componente destinado a facilitar el intercambio y la ejecución de programas de manera rápida y sencilla. Esto permitirá gestionar múltiples aplicaciones sin necesidad de recompilar el software base del sistema, abstrayendo al usuario de los procesos internos y posibilitando la ejecución de nuevas funcionalidades de forma prácticamente inmediata.

Por último, este dispositivo está pensado para estar desacoplado de cualquier dispositivo electrónico externo, por lo que se hará énfasis en el consumo energético, aspecto que se tratará más adelante.

2.2. Software

En cuanto a las necesidades del software, surgen diversos retos relacionados con la arquitectura del sistema y su capacidad de mantenimiento y escalabilidad a largo plazo. Además, es necesario contemplar la posibilidad de que usuarios avanzados deseen profundizar en el funcionamiento interno del software y adaptarlo todavía más a sus necesidades específicas.

Por este motivo, el sistema se estructurará en diferentes capas, abarcando desde componentes completamente externos al desarrollo principal hasta capas internas propias del sistema. Esta organización tiene como objetivo desacoplar al máximo las distintas funcionalidades y subsistemas que no deberían depender directamente entre sí, reduciendo así la posibilidad de errores y disminuyendo la carga de procesamiento sobre el dispositivo.

Asimismo, esta arquitectura permitirá mejorar la mantenibilidad del sistema, facilitar la escalabilidad futura e incorporar capas adicionales de desarrollo, como entornos de pruebas y validación, favoreciendo un desarrollo más robusto, modular y extensible.

Otro de los retos principales reside en abstraer la capa de desarrollo de cara al usuario. En este sentido, será necesario proporcionar una forma sencilla y accesible para que el usuario pueda desarrollar soluciones adaptadas a sus necesidades y al hardware conectado al dispositivo.

Para ello, se plantea la creación de un Domain Specific Language (DSL), diseñado específicamente para simplificar el desarrollo de aplicaciones dentro del ecosistema del dispositivo. Este lenguaje permitirá a los usuarios crear programas y definir comportamientos personalizados sin necesidad de poseer conocimientos avanzados sobre el lenguaje interno del sistema ni de consultar documentación extensa o compleja.

De esta manera, se busca reducir la barrera de entrada al desarrollo sobre la plataforma, facilitando la creación de nuevas funcionalidades y promoviendo un ecosistema más flexible, accesible y adaptable a distintos perfiles de usuario.

Todo este sistema estará basado en el lenguaje C++, aprovechando el entorno de desarrollo ESP-IDF, lo que permite un control más eficiente de los recursos y una mejor optimización del uso de memoria frente a otras alternativas como el IDE de Arduino para ESP32.

2.3. Casos de uso

Con todo lo planteado anteriormente, se puede concluir el diseño de un dispositivo que se adapta a las necesidades expuestas inicialmente. Con esta idea en mente, es posible abordar una amplia variedad de situaciones que el usuario pueda requerir.

Por ejemplo, si el usuario desea monitorizar un sistema de hidroponía, podría conectar un módulo encargado de obtener datos del entorno y acoplarlo al dispositivo. De este modo, sería posible monitorizar los datos en tiempo real o según la configuración definida por el usuario. Asimismo, también se cubren necesidades de usuarios que deseen utilizar el sistema en contextos más dinámicos, como la monitorización de rutas de senderismo mediante la incorporación de módulos GPS. Todo ello basado en un sistema plug-and-play.

Incluso se deja la puerta abierta a usuarios avanzados que deseen integrar software adicional dentro del sistema, aprovechando la arquitectura modular propuesta, la cual facilita la extensión y personalización del dispositivo de manera sencilla.

Con todo ello, se obtiene un dispositivo modular, escalable y adaptable a cualquier entorno, además de ser accesible y fácil de usar para el usuario final.

Capítulo 3

Puesta en marcha

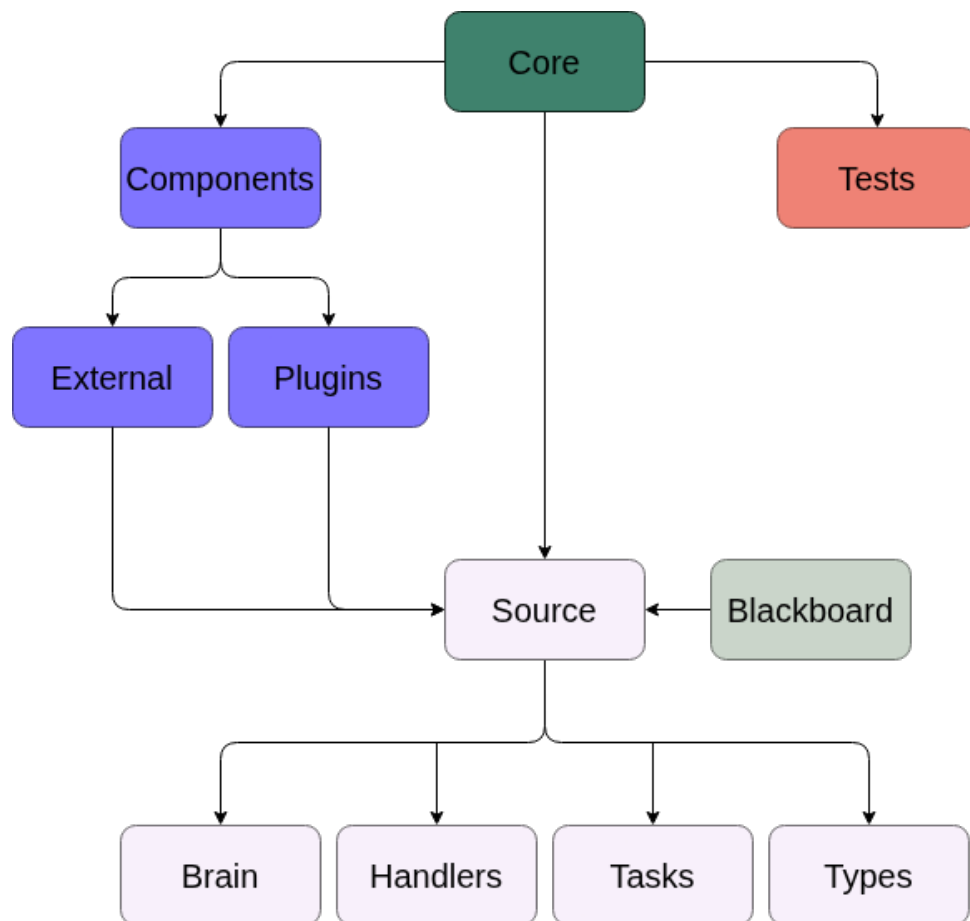


Figura 3.1: Representación conceptual del diseño del software.

Antes de abordar el funcionamiento interno del sistema, resulta necesario presentar de manera general la arquitectura software sobre la que se apoya el dispositivo. Para ello, en la Figura 3.1 se muestra un esquema simplificado de la organización interna del sistema y de las diferentes capas que lo componen.

A lo largo de las siguientes secciones se detallará el funcionamiento de cada una de estas capas, prestando especial atención a aquellas que forman parte del núcleo principal del sistema, agrupadas dentro del directorio source.

La arquitectura ha sido diseñada siguiendo una filosofía modular y desacoplada, permitiendo que gran parte de los bloques del sistema puedan funcionar de manera independiente o incluso ser sustituidos por otras tecnologías sin comprometer el funcionamiento global del dispositivo.

Entre los bloques principales de la arquitectura destacan los siguientes:

- **Components:** contiene componentes reutilizables encargados de abstraer funcionalidades específicas del sistema, como podría ser el “lexer” del que trataremos mas tarde. Su objetivo es facilitar el desarrollo y mantenimiento del software, permitiendo reutilizar lógica común en diferentes partes del proyecto. Para más información sobre estos componentes, puede consultarse el Apéndice 1.
- **Blackboard:** actúa como sistema centralizado de gestión de datos y configuración. Este bloque almacena variables compartidas y parámetros globales utilizados por el sistema, permitiendo una gestión más estructurada y simplificada del estado interno del dispositivo.
- **Tests:** engloba las pruebas destinadas a validar el correcto funcionamiento de las distintas funcionalidades del sistema. Esta capa permite detectar errores de forma temprana y facilita la mantenibilidad y robustez del software.
- **Source:** representa el núcleo principal de la lógica del sistema. Dentro de esta capa destaca especialmente el módulo brain, encargado de coordinar gran parte del comportamiento interno y de las funcionalidades principales del dispositivo.

Esta separación por capas permite desacoplar responsabilidades dentro del sistema, facilitando la escalabilidad futura, la integración de nuevas funcionalidades y la posibilidad de adaptar o sustituir componentes concretos sin afectar al resto de la arquitectura.

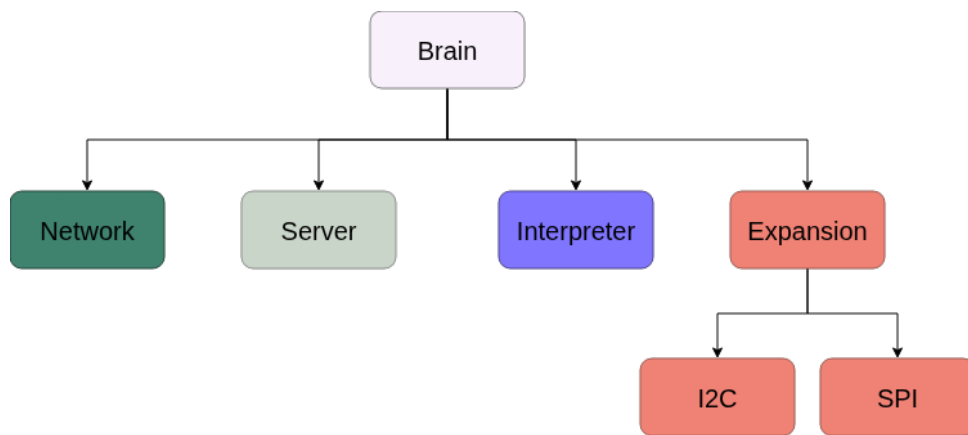


Figura 3.2: Representación conceptual del brain.

Para profundizar en el diseño del software, se utilizará como base la Figura 2, la cual representa de forma conceptual el bloque brain, ya introducido dentro de la arquitectura general mostrada en la Figura 1. Este componente constituye el núcleo principal del sistema y concentra la lógica central encargada de coordinar el comportamiento global del dispositivo.

El bloque brain actúa como elemento orquestador del sistema, gestionando la interacción entre las distintas capas y módulos descritos anteriormente. Su función principal consiste en integrar y coordinar la ejecución de los diferentes subsistemas, garantizando la coherencia del flujo de datos y del comportamiento del dispositivo en tiempo de ejecución.

En las siguientes secciones se analizará en detalle su estructura interna y su papel dentro de la arquitectura global del sistema.

3.1. Conectividad (Network)

Uno de los aspectos fundamentales del dispositivo es la conectividad inalámbrica. Como se mencionó anteriormente, la conectividad Wi-Fi representa uno de los motivos principales por los que se seleccionó el ESP32-S3 como núcleo central del sistema. Esta característica no solo permite la comunicación con otros dispositivos, sino que también facilita la configuración, monitorización y gestión remota del sistema.

Con el objetivo de adaptar el dispositivo a diferentes escenarios de uso, se plantean dos modos principales de funcionamiento relacionados con la conectividad inalámbrica: el modo Access Point (AP) y el modo Station (STA). Cada uno de ellos está orientado a cubrir necesidades específicas según el entorno en el que se utilice el dispositivo.

3.1.1. Modo Access Point (AP)

El modo Access Point está diseñado para escenarios en los que el dispositivo debe funcionar de manera autónoma, sin depender de una red Wi-Fi externa. En este modo, el dispositivo genera su propia red inalámbrica, permitiendo que el usuario pueda conectarse directamente desde un ordenador, teléfono móvil u otro dispositivo compatible.

Durante el primer arranque, el sistema crea una red Wi-Fi con unas credenciales predefinidas y asigna automáticamente una dirección IP local. Una vez conectado, el usuario puede acceder a la interfaz de configuración del dispositivo y modificar parámetros básicos, como el nombre de la red o la contraseña de acceso.

Tras completar la configuración inicial, el dispositivo cambia automáticamente a un modo de funcionamiento orientado al usuario final. Este cambio tiene como objetivo evitar modificaciones accidentales o accesos no autorizados a la configuración interna del sistema.

Para volver a habilitar el modo de configuración será necesario realizar una secuencia física de botones definida previamente. De esta forma, se añade una capa adicional de protección frente a accesos remotos no autorizados, reduciendo la posibilidad de que un atacante pueda modificar la configuración del sistema incluso en el caso de haber obtenido acceso a la red inalámbrica.

No obstante, debe tenerse en cuenta que esta protección está enfocada principalmente a ataques remotos. En caso de que un atacante obtenga acceso físico al dispositivo, podría manipular directamente el hardware y recuperar el control del sistema. Debido a ello, la seguridad física queda fuera del alcance de las medidas implementadas a nivel software.

3.1.2. Modo Station (STA)

El segundo modo de funcionamiento corresponde al modo Station (STA). En este caso, el dispositivo se conecta a una red Wi-Fi ya existente, actuando como un cliente más dentro de la infraestructura de red.

Este modo está pensado principalmente para entornos fijos o estaciones de trabajo, donde el dispositivo permanece conectado de manera permanente a una red inalámbrica. Gracias a ello, resulta posible acceder al sistema desde otros dispositivos conectados a la misma red, permitiendo consultar información, visualizar métricas o interactuar con el dashboard del dispositivo desde un ordenador u otros terminales del cual hablaremos más tarde.

Desde el punto de vista de la seguridad, este modo añade una capa de protección adicional, ya que cualquier atacante potencial necesitaría acceder previamente a la red local antes de poder intentar interactuar con el dispositivo.

De manera similar al modo AP, una vez finalizada la configuración inicial, el sistema pasa automáticamente al modo de usuario, restringiendo cambios críticos en la configuración y evitando modificaciones no autorizadas.

3.1.3. Gestión de conectividad y seguridad

La coexistencia de ambos modos de funcionamiento permite adaptar el dispositivo a diferentes contextos de uso sin comprometer la experiencia del usuario. Mientras que el modo AP facilita la portabilidad y el uso autónomo del sistema, el modo STA permite integrar el dispositivo dentro de infraestructuras de red ya existentes.

La configuración de ambos modos de funcionamiento se realiza mediante la interfaz web integrada del dispositivo, mostrada en la Figura 3.3. A través de este panel, el usuario puede gestionar los parámetros de conectividad del sistema, incluyendo las credenciales de acceso y el modo de funcionamiento activo.

Asimismo, la separación entre un modo de configuración y un modo de usuario permite desacoplar las tareas críticas de administración del funcionamiento habitual del sistema, mejorando tanto la seguridad como la estabilidad general del dispositivo.



Figura 3.3: Interfaz remota del configuración de la conectividad.

3.2. Servidor (Server)

En esta capa se aborda el sistema de servidor integrado dentro del dispositivo, el cual se apoya directamente en la capa de conectividad descrita anteriormente. En particular, el servidor se ejecuta sobre la red Wi-Fi del sistema, ya sea en modo Access Point (AP) o en modo Station (STA), permitiendo así que el usuario pueda acceder a sus funcionalidades independientemente del entorno de uso.

El objetivo principal de esta capa es centralizar la interacción entre el usuario y el dispositivo, proporcionando una interfaz unificada para la configuración, gestión y uso del sistema.

Funcionalidades del servidor

El servidor implementa varias funcionalidades principales que permiten la gestión completa del dispositivo:

Configuración de red y dispositivos

En primer lugar, el servidor permite la configuración de los parámetros de red del dispositivo, incluyendo el modo de conexión, credenciales Wi-Fi y parámetros asociados a la conectividad.

Además, en la medida de lo posible, también permite gestionar y consultar información relativa a los dispositivos o módulos conectados al sistema. Esto proporciona al usuario una visión global del estado del sistema y de sus componentes, facilitando la monitorización y depuración.

Documentación integrada

El servidor incluye una sección de documentación integrada dentro del propio dispositivo. Esta documentación contiene las especificaciones del Domain Specific Language (DSL), el cual se detallará en secciones posteriores.

El objetivo de esta funcionalidad es reducir la dependencia de fuentes externas de información, permitiendo que el usuario pueda consultar directamente desde el dispositivo todo lo necesario para desarrollar herramientas o extender su funcionalidad.

De esta forma, se unifica el entorno de desarrollo y uso en una única plataforma, reduciendo el tiempo necesario para la consulta de documentación externa y mejorando la experiencia de desarrollo.

Dashboard de control

Uno de los componentes principales del servidor es el dashboard el cual podemos ver en la Figura 3.4, que actúa como interfaz remota del sistema.

Este panel permite al usuario:

- Crear programas o herramientas
- Editarlos
- Eliminarlos
- Gestionar su ejecución
- Consultar registros generados por las aplicaciones

Estos registros permiten monitorizar el comportamiento de los programas desarrollados por el usuario, facilitando la depuración y el análisis de datos recogidos por el propio sistema.

El objetivo del dashboard es proporcionar una interfaz completa que permita al usuario gestionar el dispositivo sin necesidad de herramientas externas, consolidando así el enfoque de autonomía del sistema.

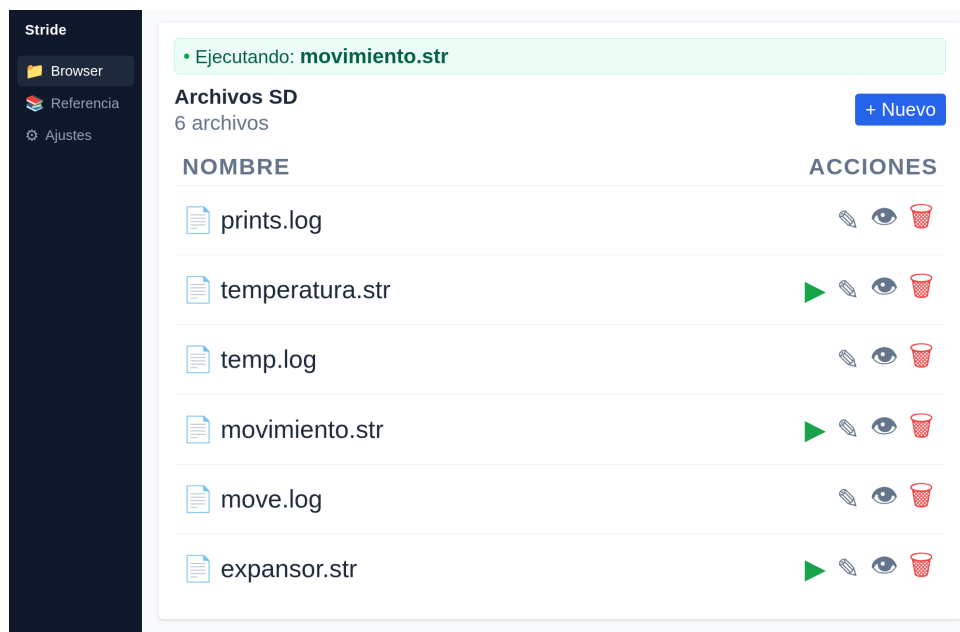


Figura 3.4: Interfaz remota del dashboard.

Independencia de herramientas externas

Uno de los objetivos fundamentales de esta capa es evitar la dependencia de software externo o de dispositivos adicionales. El sistema está diseñado para permitir que el usuario pueda configurar, programar y gestionar sus aplicaciones directamente desde el propio dispositivo.

Esto resulta especialmente relevante en escenarios donde no se dispone de conexión a Internet o acceso a herramientas de desarrollo externas. En estos casos, el dispositivo actúa como una plataforma autónoma completamente funcional.

Integración con los modos del sistema

El servidor se integra directamente con los distintos modos de funcionamiento del ESP32 descritos anteriormente (modo AP y modo STA), adaptando su comportamiento en función del estado del sistema.

En particular, el servidor expone diferentes rutas y funcionalidades según el modo en el que se encuentre el dispositivo. Esto permite aislar correctamente las funciones críticas de configuración respecto a las funciones de usuario.

Por ejemplo, en modo configuración se habilitan rutas específicas destinadas a la gestión de red y parámetros del sistema. Una vez finalizada la configuración, el sistema cambia automáticamente al modo de usuario, restringiendo el acceso a dichas rutas para evitar modificaciones no autorizadas.

Este mecanismo no solo mejora la seguridad del sistema, sino que también facilita la extensibilidad del servidor, ya que permite añadir nuevas rutas o funcionalidades de forma modular, sin comprometer la arquitectura existente.

Una vez definida la capa de interacción y gestión del sistema mediante el servidor, resulta necesario abordar cómo el usuario puede desarrollar e integrar nuevas funcionalidades dentro del dispositivo.

3.3. Interpreter

La siguiente capa del sistema está orientada a facilitar el desarrollo de herramientas y aplicaciones por parte del usuario. Para ello, se implementa un sistema de interpretación dinámica basado en un Domain Specific Language (DSL), cuyo objetivo principal es simplificar la interacción con el hardware y reducir la complejidad del desarrollo interno del dispositivo y del cual abordaremos a continuación.

Esta capa hace uso de uno de los componentes principales mencionados anteriormente: el “lexer”. Dicho componente es el encargado de analizar y tokenizar las instrucciones escritas por el usuario dentro del DSL.

El proceso de tokenización permite transformar las instrucciones del lenguaje en estructuras comprensibles para el sistema, verificando además que su sintaxis sea válida antes de continuar con el proceso de interpretación.

Una vez generados los tokens, estos son procesados por el “interpreter”, cuya función principal consiste en analizar secuencialmente las instrucciones recibidas y traducirlas a operaciones internas equivalentes en C++. De esta manera, el DSL actúa como una capa de abstracción sobre el lenguaje nativo del sistema, permitiendo simplificar el desarrollo de aplicaciones sin exponer al usuario a la complejidad interna de la plataforma.

El conjunto de instrucciones soportadas por el intérprete se define mediante un lenguaje específico de dominio (Domain Specific Language, DSL), diseñado específicamente para este sistema con el objetivo de simplificar la interacción con el hardware y abstraer la complejidad de la implementación interna.

La especificación completa de este lenguaje, incluyendo su sintaxis, estructura y funcionalidades, puede consultarse en el Apéndice 2.

El funcionamiento general del sistema puede resumirse en las siguientes etapas:

- Escritura de instrucciones mediante el DSL.
- Tokenización y validación sintáctica mediante el “lexer”.
- Interpretación de tokens mediante el “interpreter”.
- Traducción de instrucciones a operaciones internas del sistema.
- Ejecución de funcionalidades sobre las diferentes capas hardware y software.

El principal objetivo de esta arquitectura es reducir la complejidad del desarrollo desde el punto de vista del usuario, especialmente en tareas relacionadas con la interacción con módulos externos y sistemas de expansión hardware.

Para ello, el “interpreter” se apoya directamente en la capa de expansión, encargada de abstraer la comunicación con el hardware externo conectado al dispositivo.

Para permitir que las herramientas desarrolladas por el usuario puedan interactuar con hardware externo de forma transparente, el sistema incorpora una capa de expansión encargada de abstraer la comunicación con los distintos periféricos conectados.

3.4. Expansión (Expansion)

La capa de expansión es la encargada de gestionar la comunicación entre el núcleo principal del sistema y los distintos módulos hardware externos conectados al dispositivo.

Con el objetivo de mantener una arquitectura modular y desacoplada, esta capa abstrae la complejidad asociada a la comunicación hardware, proporcionando interfaces unificadas para los distintos buses utilizados por el sistema.

Debido a la naturaleza de los dispositivos conectados, la capa de expansión se divide en dos subsistemas principales:

- Gestión del bus SPI
- Gestión del bus I2C

Esta organización puede observarse de forma esquemática en la Figura 3.5, donde se representa la estructura general de la capa de expansión y la interacción entre los distintos buses de comunicación utilizados por el sistema.

Esta separación permite aislar responsabilidades y adaptar el funcionamiento de cada protocolo a las necesidades específicas de los distintos módulos conectados al sistema.

Gestión del bus SPI

La capa Bus SPI es la encargada de administrar todos los dispositivos que hacen uso del protocolo SPI dentro del sistema.

Este bus se utiliza principalmente para periféricos de alta velocidad, permitiendo compartir las líneas principales de comunicación entre varios dispositivos mediante mecanismos de selección individuales.

Para ello, cada módulo conectado dispone de una línea de selección propia (Chip Select), permitiendo habilitar únicamente el dispositivo con el que se desea establecer comunicación en cada momento.

Gracias a esta arquitectura, múltiples módulos pueden coexistir sobre el mismo bus físico sin generar conflictos de comunicación, manteniendo una estructura eficiente y escalable.

La capa Bus SPI abstrae toda esta lógica de gestión, permitiendo que el resto del sistema interactúe con los dispositivos conectados sin necesidad de gestionar manualmente los mecanismos de selección del bus.

Gestión del bus I2C

Por otro lado, la capa Bus I2C se encarga de gestionar los dispositivos conectados mediante el protocolo I2C.

Este bus está orientado principalmente a módulos de expansión externos y periféricos de control, proporcionando una interfaz sencilla para la comunicación entre múltiples dispositivos conectados al sistema.

A diferencia del bus SPI, el protocolo I2C puede presentar conflictos derivados de la reutilización de direcciones entre dispositivos. Debido a ello, el diseño original del sistema contemplaba la incorporación

de un multiplexor I2C dedicado que permitiera ampliar la capacidad de conexión y aislar correctamente los distintos módulos.

Sin embargo, debido a las limitaciones del prototipo actual, se optó por implementar una solución híbrida basada en software y hardware.

Esta aproximación combina:

- Los buses I2C nativos del sistema.
- Sistemas de expansión de pines digitales.
- Mecanismos de control implementados mediante software.

Gracias a esta solución, es posible emular parcialmente el comportamiento de un sistema de multiplexación, permitiendo gestionar distintos dispositivos y configuraciones dentro del bus I2C.

No obstante, esta implementación presenta ciertas limitaciones respecto al diseño inicialmente planteado, especialmente en términos de escalabilidad y capacidad de expansión hardware, aspectos que serán tratados más adelante.

La capa Bus I2C también es responsable de gestionar las distintas direcciones de comunicación necesarias para garantizar el correcto funcionamiento de todos los dispositivos conectados al sistema, evitando conflictos y manteniendo una comunicación estable y organizada entre los diferentes módulos.

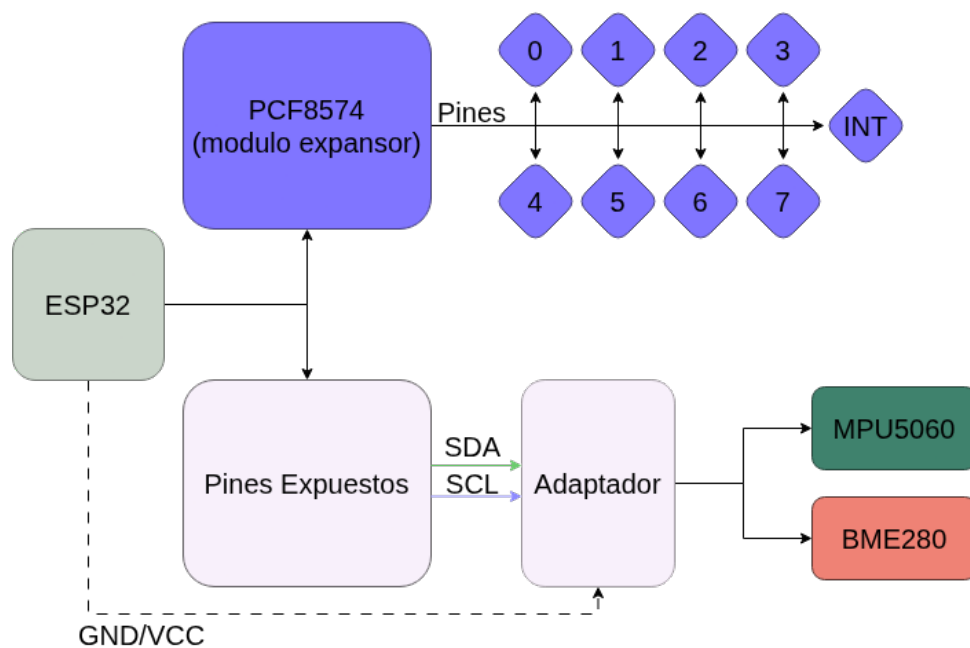


Figura 3.5: Representación conceptual de las conexiones.

3.5. Display

Por último, se describe el funcionamiento del sistema de visualización (display), el cual constituye la principal capa de interacción entre el usuario y el dispositivo de carácter físico. A través de este

módulo, el usuario puede obtener información del sistema y controlar su funcionamiento de forma directa.

Para la implementación de esta capa se ha utilizado un componente adicional incluido en el bloque de components, encargado de implementar y gestionar una máquina de estados. Esta máquina de estados permite determinar en todo momento el estado actual del sistema de visualización, facilitando la organización de la interfaz y permitiendo habilitar o restringir funcionalidades en función del contexto.

Gracias a este enfoque, el sistema puede adaptar su comportamiento de forma dinámica según el estado en el que se encuentre el dispositivo.

Estados del sistema de visualización

El sistema de display se compone de varios estados principales, cada uno con una funcionalidad específica dentro de la interfaz del dispositivo:

Startup

Este estado corresponde a la fase inicial del sistema. Su función es principalmente visual y de retroalimentación, indicando al usuario que el dispositivo está en proceso de arranque.

Aunque desde el punto de vista del usuario se trata de una pantalla meramente informativa, internamente el sistema aprovecha este estado para inicializar los distintos módulos y servicios del dispositivo.

Main

Este es el estado principal del sistema y actúa como pantalla de inicio del dispositivo.

En este estado se muestra la información más relevante para el usuario, entre la que se incluye:

- La dirección IP asignada al dispositivo, en función del modo de conectividad activo (modo AP o modo STA).
- La lista de herramientas o programas disponibles en el sistema.
- El acceso a los archivos de registros generados por las distintas aplicaciones.

Desde este estado, el usuario puede seleccionar qué programas ejecutar o acceder a los registros generados por el sistema.

Running

El estado Running se activa cuando un programa o herramienta se encuentra en ejecución.

Su principal función es proporcionar retroalimentación al usuario sobre el estado del proceso en curso, mostrando información como el tiempo de ejecución o el estado general del programa.

Este estado permite mantener al usuario informado sin necesidad de abandonar el flujo de ejecución del sistema.

View

El estado View está destinado a la visualización de archivos de log.

En este modo, el usuario puede consultar la información generada por los programas, así como desplazarse por el contenido en caso de que los registros sean extensos.

Este estado facilita el análisis y depuración de las aplicaciones desarrolladas dentro del sistema.

Una vez descritas las principales capas software del sistema y el mecanismo mediante el cual el dispositivo gestiona la conectividad, interpretación y expansión de funcionalidades, resulta necesario abordar los aspectos hardware que permiten dar soporte físico a toda la arquitectura planteada.

En particular, uno de los elementos más críticos dentro del diseño del dispositivo es el sistema de alimentación, ya que el objetivo principal del proyecto es desarrollar una plataforma portátil, autónoma y reutilizable en distintos entornos de uso.

3.6. Tasks (Gestión de tareas)

Una vez descrito el funcionamiento del bloque brain, es necesario introducir un mecanismo fundamental sobre el que este se apoya para la ejecución concurrente de funcionalidades: el sistema de tareas del dispositivo el cual podemos observar en la Figura 3.6.

Otra de las funcionalidades clave aprovechadas del ESP32 es el sistema de tareas proporcionado por FreeRTOS. Este mecanismo permite aislar las distintas responsabilidades del sistema en tareas independientes, mejorando tanto la organización interna del software como la estabilidad global del dispositivo.

Tal y como se ha mencionado anteriormente, el ESP32-S3 dispone de una arquitectura de doble núcleo. Esta característica resulta especialmente relevante dentro del diseño propuesto, ya que permite distribuir las tareas del sistema entre ambos núcleos, separando aquellas operaciones críticas de fondo de las tareas orientadas a la interacción con el usuario.

Gracias a esta división, se evita que operaciones bloqueantes, como el acceso a almacenamiento o la inicialización de periféricos, afecten a la capacidad de respuesta del sistema, lo que aporta una mayor robustez frente a bloqueos o tiempos de espera prolongados.

Desde esta perspectiva, el bloque brain no ejecuta todas las operaciones de forma secuencial, sino que delega gran parte del comportamiento del sistema en tareas independientes gestionadas por el planificador de FreeRTOS.

En el diseño actual, las principales tareas del sistema se distribuyen de la siguiente manera:

- OpenCard: encargada de la inicialización y verificación de la tarjeta SD. Esta tarea valida que el sistema de almacenamiento esté disponible antes de permitir el acceso al resto de funcionalidades. Se ejecuta en el Core 0.
- ServerMode: responsable de gestionar la entrada física asociada al cambio de modo del servidor, permitiendo alternar entre los distintos estados de operación del dispositivo sin interferir con el flujo principal. Se ejecuta en el Core 0.
- ReadCard: encargada de la lectura de los datos almacenados en la tarjeta SD una vez que la tarea OpenCard ha finalizado correctamente la inicialización. Se ejecuta en el Core 1.
- SelectProgram: gestiona las entradas del usuario relacionadas con la selección y ejecución de programas o herramientas dentro del sistema. Se ejecuta en el Core 1.
- Display: responsable de la actualización de la interfaz gráfica del dispositivo, mostrando información en tiempo real al usuario. Esta tarea está estrechamente ligada a la máquina de estados del sistema de visualización y se desarrollará con mayor detalle en la sección correspondiente. Se ejecuta en el Core 1.

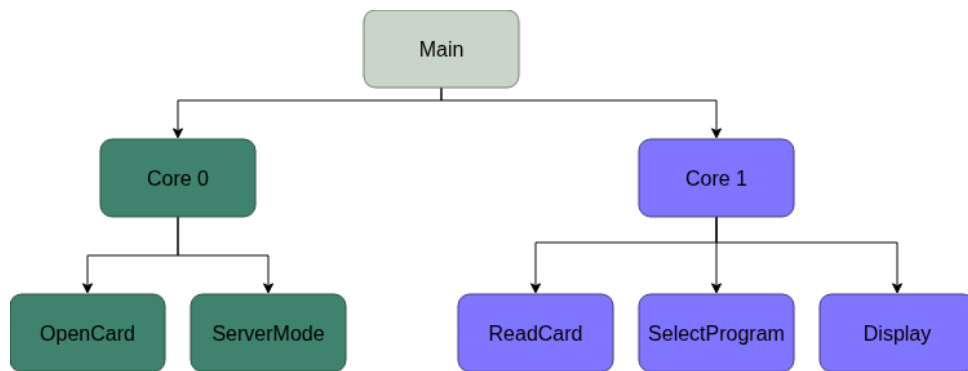


Figura 3.6: Representación conceptual de las tasks.

Esta distribución asigna el Core 0 a tareas de control y gestión de bajo nivel, mientras que el Core 1 se encarga de la interacción con el usuario y la representación visual del sistema. De este modo, operaciones críticas como la lectura de la tarjeta SD o la gestión de modos del servidor no interfieren con la fluidez de la interfaz.

Aunque todas las tareas comparten un nivel similar de prioridad dentro del planificador de FreeRTOS, no todas requieren la misma asignación de recursos. En particular, las tareas relacionadas con el acceso a la tarjeta SD (OpenCard y especialmente ReadCard) disponen de un mayor tamaño de pila, debido a la complejidad de las operaciones de lectura, gestión del sistema de archivos y procesamiento de datos.

Este dimensionamiento responde a la importancia crítica del subsistema de almacenamiento dentro del funcionamiento global del dispositivo, ya que gran parte de las funcionalidades dependen directamente de la correcta lectura de la tarjeta SD.

En conjunto, esta arquitectura basada en tareas concurrentes permite que el sistema mantenga un comportamiento estable y reactivo incluso bajo cargas de trabajo elevadas, mejorando significativamente la experiencia de uso del dispositivo y reforzando la separación de responsabilidades introducida en el bloque brain.

Capítulo 4

Gestión energética y sistema de alimentación

Uno de los aspectos más importantes dentro del diseño hardware del dispositivo es el sistema de alimentación. Debido a que el objetivo del proyecto es desarrollar una plataforma portátil y desacoplada de dispositivos externos, resulta fundamental garantizar una autonomía suficiente para distintos escenarios de uso.

Para ello, se ha realizado un estudio energético basado en los componentes ideales contemplados para el sistema final, teniendo en cuenta tanto el consumo individual de cada módulo como los diferentes modos de funcionamiento del dispositivo.

No obstante, debido a las limitaciones de material y tiempo del prototipo actual, estos cálculos no han sido implementados físicamente en esta versión del dispositivo. Aun así, el estudio realizado permite establecer una base sólida para futuros prototipos y definir los requisitos necesarios para integrar correctamente un sistema de alimentación autónomo.

4.1. Sistemas de baterías

Sistema de baterías El sistema planteado hace uso de baterías de litio debido a su elevada densidad energética y a su reducido tamaño, características especialmente relevantes en dispositivos portátiles.

Sin embargo, este tipo de baterías requiere una gestión cuidadosa tanto durante la carga como durante el funcionamiento, ya que presentan una alta sensibilidad frente a situaciones de:

- Sobretensión
- Subtensión
- Sobrecorriente
- Cortocircuitos
- Desequilibrios entre celdas

Debido a ello, el sistema incorpora un módulo de protección encargado de supervisar continuamente el estado de las celdas y garantizar un funcionamiento seguro.

Además, el sistema debe asegurar que todas las celdas mantengan niveles de carga equilibrados para evitar degradación prematura o daños irreversibles en las baterías. Para ello, se plantea el uso de un sistema de balanceo o ecualización de carga.

En el caso de utilizar celdas pertenecientes al mismo fabricante y lote de producción, este equilibrio puede gestionarse mediante módulos de protección convencionales. Sin embargo, cuando se utilizan baterías con características distintas, resulta necesario incorporar sistemas de balanceo más avanzados capaces de redistribuir correctamente la carga entre las diferentes celdas.

El objetivo de estas medidas es maximizar la vida útil de las baterías y garantizar un funcionamiento estable y seguro del dispositivo.

4.2. Sistema de carga

Junto al sistema de protección, el dispositivo incorpora un módulo de carga encargado de alimentar las celdas de litio utilizando los algoritmos de carga adecuados para este tipo de baterías.

Este módulo recibe la tensión de entrada y gestiona automáticamente las distintas fases de carga necesarias para mantener la integridad y seguridad de las celdas.

Debido a la configuración energética planteada para el sistema, resulta necesario utilizar módulos compatibles con configuraciones multicelda.

4.3. Estimación de consumo y autonomía

A partir de los cálculos de consumo realizados sobre los distintos componentes del sistema, se estimó una configuración basada en tres celdas de litio de aproximadamente 3000 mAh cada una.

Tabla 4.1: Consumo estimado de los componentes del sistema

Componente	Mínimo (mA)	Máximo (mA)	Notas
Microcontrolador (ESP32)	0.007	340	Wi-Fi activo en transmisión máxima.
Pantalla (controlador)	0.1	6	Sin retroiluminación.
Retroiluminación pantalla	0	40	Dependiente del brillo (PWM).
Sensor inercial (IMU)	0.1	15	Según la tasa de muestreo.
Almacenamiento (microSD)	0.05	100	Picos de consumo durante escritura.
Gestión de alimentación	0.01	0.02	Consumo quiescente.
Expansor de pines	0.1	1	Dependiente del uso.
LEDs indicadores (x3)	0	30	Dependiente del estado.
Zumbador	0	30	Solo durante activación.
Total estimado	≈ 0.4	≈ 560–590	

Teniendo en cuenta esta configuración, la autonomía estimada del dispositivo se sitúa aproximadamente entre 22 y 24 horas de funcionamiento, dependiendo del modo de uso y de los periféricos conectados al sistema.

Debido a esta configuración de tres celdas, el sistema requiere:

- Un módulo de carga tipo 3S, encargado de gestionar la carga simultánea de las tres baterías.
- Un módulo de protección 3S, destinado a supervisar y proteger las celdas frente a posibles fallos eléctricos.

Esta arquitectura permite establecer una base energética suficientemente robusta para futuros prototipos del dispositivo, manteniendo el enfoque portátil y autónomo planteado inicialmente.

4.4. Sistema de refrigeración y control térmico

Con el objetivo de aumentar la vida útil del dispositivo y garantizar un funcionamiento estable bajo distintos escenarios de carga, se plantea la incorporación de un sistema de refrigeración activo basado en un ventilador.

Este sistema tiene como finalidad evitar el sobrecalentamiento de los componentes electrónicos, especialmente durante situaciones de alta demanda computacional o durante la carga de las baterías, donde el incremento térmico puede afectar tanto al rendimiento como a la seguridad del sistema.

El control del ventilador se realiza de forma independiente respecto a la lógica principal del dispositivo, siguiendo el mismo principio de desacoplamiento aplicado en el resto de la arquitectura del sistema. De este modo, su funcionamiento no interfiere con la ejecución del software principal ni con las capas superiores del sistema.

Para su activación y desactivación se plantea el uso de un sensor de temperatura, el cual permite monitorizar las condiciones térmicas internas del dispositivo. En función de los umbrales definidos, el sistema puede activar automáticamente el ventilador cuando se detecten temperaturas elevadas y desactivarlo cuando el sistema vuelva a rangos de funcionamiento seguros.

De esta forma, se introduce una capa adicional de protección térmica que complementa el sistema de gestión energética descrito anteriormente, contribuyendo a la estabilidad global del dispositivo.

No obstante, al igual que ocurre con otros elementos avanzados del sistema de alimentación, tanto el sistema de refrigeración como el mecanismo de encendido y apagado no han sido implementados en el prototipo actual debido a las limitaciones de hardware disponibles. Aun así, ambos elementos se incluyen dentro del diseño conceptual del sistema final, sirviendo como referencia para futuras iteraciones del dispositivo.

En particular, el sistema de encendido y apagado se plantea como un mecanismo físico de control de energía del dispositivo, permitiendo gestionar su activación de forma independiente. Sin embargo, en el prototipo actual este comportamiento no ha sido necesario, ya que el sistema únicamente funciona cuando se encuentra conectado a una fuente de alimentación externa, sin disponer de alimentación autónoma mediante baterías.

Capítulo 5

Módulos conectados al dispositivo

Una vez definidas las diferentes capas software y la arquitectura hardware general del sistema, resulta necesario describir los módulos específicos utilizados dentro del prototipo desarrollado.

Estos componentes permiten validar tanto la arquitectura modular planteada como la capacidad de expansión e interacción del dispositivo con distintos periféricos externos.

5.0.1. Sistema de interacción con el usuario

Para facilitar la interacción directa con el dispositivo, el sistema incorpora una pantalla TFT como interfaz principal de visualización.

Idealmente, esta pantalla debería disponer de capacidades táctiles, permitiendo al usuario interactuar con el sistema mediante una interfaz similar a la utilizada en dispositivos móviles convencionales. Esto reduciría considerablemente la curva de aprendizaje y mejoraría la experiencia de uso del dispositivo.

No obstante, debido a las limitaciones del prototipo actual, la pantalla utilizada no dispone de funcionalidades táctiles. Como solución alternativa, se implementó un sistema de navegación basado en tres botones físicos, encargados de facilitar la interacción básica con el sistema.

5.1. Sistema de botones

Cada uno de los botones implementados cumple una función específica dentro del dispositivo:

- Botón centro: permite desplazarse hacia arriba dentro de la interfaz principal del sistema.
- Botón derecha: permite desplazarse hacia abajo dentro de la interfaz. Además, este botón también permite alternar entre los distintos modos de funcionamiento del dispositivo descritos anteriormente en la capa de conectividad, concretamente entre el modo configuración y el modo usuario.
- Botón izquierdo: se utiliza para ejecutar el programa seleccionado dentro del sistema y para regresar al menú principal cuando el usuario se encuentra visualizando archivos de registros o información secundaria.

Gracias a esta configuración, es posible operar el dispositivo de forma completamente autónoma sin necesidad de periféricos externos adicionales.

5.1.1. Sistema de señalización LED

El dispositivo incorpora además tres LEDs destinados a proporcionar información visual rápida sobre el estado interno del sistema.

De derecha a izquierda, cada LED representa lo siguiente:

- LED de conectividad: indica si el dispositivo se encuentra conectado correctamente a una red Wi-Fi.
- LED de estado: muestra el modo de funcionamiento actual del sistema, permitiendo identificar visualmente si el dispositivo se encuentra en modo configuración o modo usuario.
- LED programable: actúa como pin de prueba configurable por el usuario. Este LED puede activarse o desactivarse dinámicamente mediante instrucciones definidas dentro del DSL, permitiendo validar funcionalidades o generar señales visuales personalizadas.

Este sistema de indicadores permite obtener información básica del estado del dispositivo de forma inmediata y sin necesidad de acceder al dashboard o a herramientas externas.

5.2. Módulo de expansión hardware

Tal y como se mencionó anteriormente en la capa de expansión, el sistema incorpora un módulo expansor encargado de aumentar las capacidades de entrada y salida del dispositivo.

Para ello, se utiliza un expansor de propósito general conectado mediante el bus I2C, el cual permite exponer múltiples pines digitales adicionales al usuario. Gracias a este sistema, resulta posible conectar periféricos externos y ampliar las capacidades del dispositivo sin modificar directamente el hardware principal del sistema.

Además, el bus I2C del expansor trabaja en paralelo con las conexiones destinadas a módulos externos, permitiendo emular parcialmente el sistema de expansión modular planteado inicialmente dentro de la arquitectura del proyecto.

Esta aproximación permite validar el enfoque modular y flexible del dispositivo, manteniendo la capacidad de integrar nuevos sensores o periféricos de forma relativamente sencilla.

5.3. Sensores utilizados en el prototipo

Con el objetivo de validar el funcionamiento de la arquitectura planteada y demostrar la flexibilidad del sistema, el prototipo incorpora distintos sensores conectados mediante la capa de expansión.

Entre ellos destacan:

- Un sensor ambiental encargado de obtener datos relacionados con temperatura y condiciones atmosféricas.
- Una Unidad de Medición Inercial (IMU) de 6 grados de libertad, capaz de combinar: un acelerómetro de 3 ejes, y un giroscopio de 3 ejes.

La incorporación de estos sensores permite demostrar la capacidad del sistema para adaptarse a diferentes escenarios de uso, incluyendo monitorización ambiental, análisis de movimiento y adquisición de datos en tiempo real.

Asimismo, estos módulos sirven como base de validación para las funcionalidades expuestas a través del DSL y del sistema de expansión hardware desarrollado.

Capítulo 6

Diseño final

Con todo lo expuesto anteriormente, se puede concluir el diseño de un dispositivo que se adapta a una amplia variedad de necesidades, ya que cubre un conjunto amplio de posibles casos de uso por parte del usuario.

No obstante, queda por definir el diseño físico del sistema, es decir, la disposición de los distintos componentes en la estructura final del dispositivo.

Para ello, se incluyen dos representaciones gráficas:

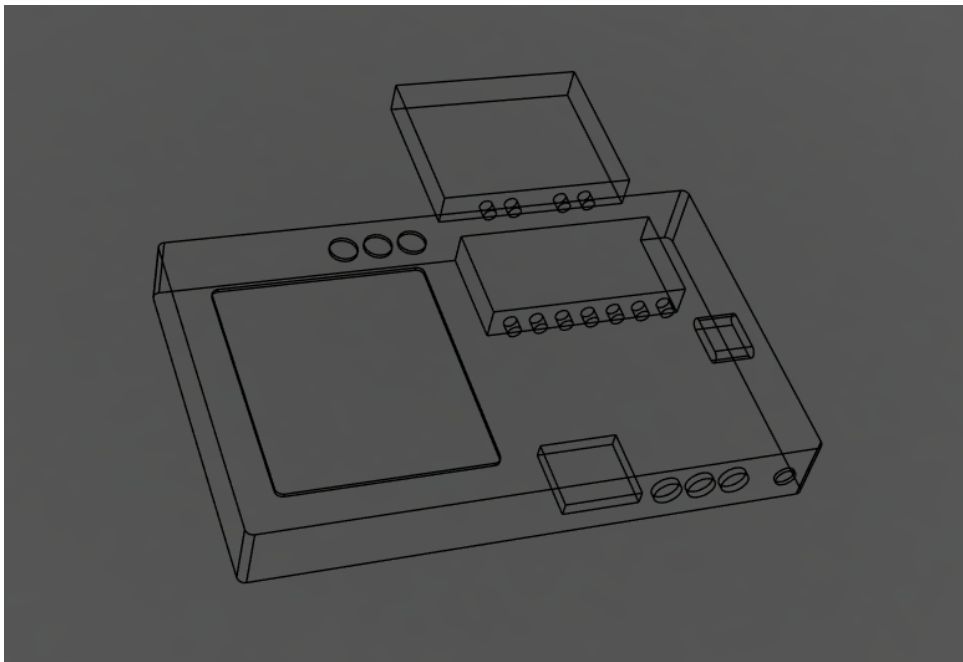


Figura 6.1: Diseño exterior del dispositivo.

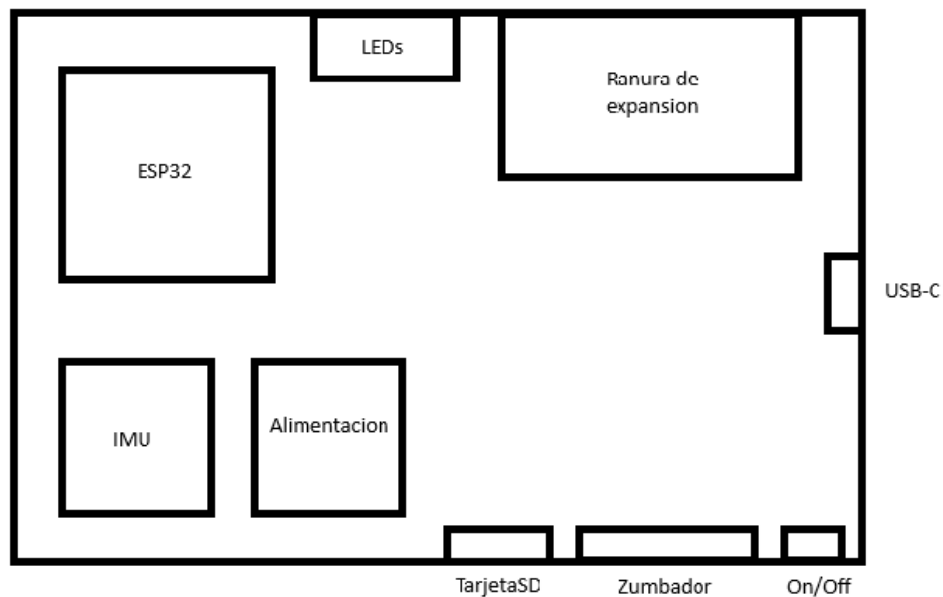


Figura 6.2: Distribución interna de los componentes.

6.1. Distribución física del dispositivo

Como puede observarse, el dispositivo integra los diferentes componentes descritos a lo largo del documento, con la excepción de los sistemas de botones físicos, que han sido integrados en la parte frontal del diseño.

En la parte izquierda del dispositivo se encuentra la pantalla principal (display), junto con los indicadores LED de estado. Estos LEDs permiten señalar información relevante del sistema, tal y como se describió en el apartado de Sistema de señalización LED, facilitando una visualización rápida del estado operativo del dispositivo.

En la zona inferior derecha se ubica el módulo de almacenamiento mediante tarjeta SD, junto con las ranuras de salida de sonido correspondientes al buzzer integrado en el sistema. Adyacente a esta zona se encuentra el botón físico de encendido y apagado del dispositivo, el cual está directamente relacionado con el sistema de gestión energética descrito en el apartado de Gestión energética y sistema de alimentación.

En el lateral derecho del dispositivo se sitúa el puerto de carga, encargado de la alimentación del sistema y de la recarga del paquete de baterías de litio.

En la parte superior se encuentra el módulo de expansión, destinado a la conexión de periféricos externos. Este módulo implementa la arquitectura de expansión descrita en la sección correspondiente, permitiendo la integración de nuevos sensores y dispositivos de forma modular y rápida.

Finalmente, la parte trasera del dispositivo incorpora ranuras de ventilación, diseñadas para facilitar la disipación térmica y garantizar un flujo de aire adecuado. Este sistema de ventilación resulta especialmente relevante durante procesos de carga y en escenarios de alta demanda energética, donde el consumo del sistema es mayor.

6.2. Coherencia con la arquitectura del sistema

El diseño físico propuesto refleja directamente la arquitectura modular descrita en las secciones anteriores, manteniendo la separación entre:

- Interfaz de usuario (display y controles)
- Sistema de expansión hardware
- Sistema de almacenamiento y registro
- Gestión energética
- Sistema de alimentación y carga

De este modo, la disposición de los componentes no solo responde a criterios de ergonomía y usabilidad, sino también a la estructura lógica del sistema, garantizando la coherencia entre el diseño hardware y la arquitectura software presentada previamente.

Capítulo 7

Resumen y Conclusiones / Summary and Conclusions

7.1. Resumen

Este trabajo ha abordado el diseño e implementación de un dispositivo IoT modular basado en el microcontrolador ESP32-S3, cuyo objetivo principal es reducir la complejidad asociada al desarrollo de sistemas embebidos y facilitar la creación rápida de soluciones personalizadas.

La arquitectura propuesta se fundamenta en un diseño en capas y completamente modular, que permite la separación clara de responsabilidades entre los distintos subsistemas del dispositivo. Esta estructura facilita la escalabilidad del sistema y permite la integración o sustitución de componentes sin afectar al funcionamiento global.

Uno de los principales aportes del proyecto es el desarrollo de un lenguaje específico de dominio (DSL), diseñado para simplificar la interacción con el hardware. Este lenguaje se apoya en una arquitectura lexer–interpreter, permitiendo la ejecución de programas definidos por el usuario sin necesidad de recompilar el firmware. El DSL incluye soporte para definición de dispositivos, control de flujo, operaciones aritméticas y bit a bit, gestión de temporización, comunicación I2C, y generación de logs.

Además, el sistema incorpora un servidor embebido que proporciona una interfaz web accesible desde navegador, permitiendo la gestión del dispositivo, edición de programas, visualización de logs y acceso a documentación integrada. A diferencia de arquitecturas tradicionales basadas únicamente en intercambio de datos, este enfoque permite una experiencia completamente autónoma desde el propio dispositivo.

Desde el punto de vista del software, el sistema integra componentes clave como el gestor de logs, el sistema de observación de variables, el lexer, el intérprete y la capa de abstracción de hardware. Asimismo, se han incorporado pruebas automatizadas para garantizar la estabilidad del núcleo del sistema.

También se ha realizado un análisis teórico del consumo energético, considerando diferentes modos de operación del dispositivo, lo que permite estimar su autonomía en escenarios portátiles con alimentación mediante baterías de litio.

Finalmente, se ha definido un diseño físico conceptual del dispositivo, orientado a la modularidad, la expansión de hardware y la optimización del consumo energético.

Es importante destacar que el prototipo implementado corresponde a una versión funcional basada en protoboard, debido a las limitaciones de tiempo y recursos propias de un Trabajo de Fin de Grado.

7.2. Conclusiones

Los objetivos planteados inicialmente en este proyecto han sido alcanzados satisfactoriamente. El sistema desarrollado proporciona una plataforma IoT modular y reutilizable capaz de adaptarse a distintos escenarios sin necesidad de modificaciones significativas en hardware o firmware.

El desarrollo del DSL ha demostrado ser una solución eficaz para reducir la complejidad del desarrollo en sistemas embebidos, proporcionando un mecanismo intuitivo de interacción con el hardware. Su arquitectura basada en intérprete permite además su futura expansión sin comprometer el núcleo del sistema.

Del mismo modo, la arquitectura modular del software ha demostrado ser clave para mejorar la mantenibilidad y escalabilidad del sistema, permitiendo una clara separación de responsabilidades entre componentes.

En cuanto al consumo energético, el análisis realizado confirma la viabilidad del sistema para aplicaciones portátiles, especialmente cuando se combinan estrategias de bajo consumo como la desactivación de periféricos y modos de suspensión.

En conjunto, este trabajo establece una base sólida para la evolución futura del sistema, tanto a nivel de hardware como de software, demostrando la viabilidad de construir plataformas IoT flexibles utilizando hardware de bajo coste.

7.3. Summary

This project has focused on the design and implementation of a modular IoT device based on the ESP32-S3 microcontroller, aimed at reducing the complexity associated with embedded system development and enabling rapid prototyping of customizable solutions.

The proposed architecture follows a fully modular, layered design that clearly separates the responsibilities of each subsystem. This structure improves scalability and allows components to be replaced or extended without affecting the overall system behaviour.

A key contribution of this work is the development of a domain-specific language (DSL) named Stride, designed to simplify hardware interaction. The language is based on a lexer–interpreter architecture, enabling user-defined programs to be executed without firmware recompilation. The DSL supports device declaration, control flow, arithmetic and bitwise operations, timing management, I2C communication, and logging capabilities.

In addition, the system integrates an embedded web server that provides a browser-accessible interface for device configuration, program management, log inspection, and integrated documentation access. Unlike traditional data-only embedded interfaces, this approach delivers a fully self-contained user experience directly on the device.

From a software perspective, the system includes core components such as a logging system, an observable variable mechanism, a lexer, an interpreter, and a hardware abstraction layer. Automated tests have also been integrated to ensure system reliability and maintainability.

A theoretical energy consumption analysis has been conducted, considering different operating modes of the device, allowing an estimation of battery life in portable usage scenarios.

Finally, a conceptual hardware design has been proposed, focusing on modularity, hardware expansion, and energy efficiency.

It should be noted that the implemented prototype is based on a breadboard setup due to the technical and time constraints of a Final Degree Project.

7.4. Conclusions

The objectives initially defined in this project have been successfully achieved. The resulting system provides a reusable and modular IoT platform capable of adapting to different application scenarios without requiring significant hardware or firmware modifications.

The developed DSL has proven to be an effective solution for reducing complexity in embedded system development, offering an intuitive interface for hardware interaction. Its interpreter-based architecture also enables future extensions without impacting the core system.

Similarly, the modular software architecture has demonstrated clear benefits in terms of maintainability and scalability, enabling a strong separation of concerns between system components.

Regarding energy consumption, the analysis confirms the feasibility of the platform for portable applications, particularly when combined with low-power strategies such as peripheral disabling and sleep modes.

Overall, this work establishes a solid foundation for the future evolution of the system at both hardware and software levels, demonstrating the viability of building flexible IoT platforms using low-cost embedded hardware.

7.5. Trabajo Futuro

Aunque el sistema desarrollado cumple los objetivos principales del proyecto, se han identificado varias líneas de mejora futura:

Extensiones del DSL

- Implementación de ejecución basada en eventos para reducir el uso de CPU.
- Incorporación de subrutinas definidas por el usuario.
- Soporte para nuevos tipos de datos (strings, arrays y estructuras simples).
- Acceso directo a periféricos internos del ESP32-S3 (ADC, sensores internos).
- Soporte para comunicación SPI además de I2C.

Evolución del hardware

- Diseño e implementación de una PCB personalizada que sustituya el prototipo en protoboard.
- Integración del sistema completo en un formato compacto y portátil.
- Validación completa del sistema de alimentación y gestión térmica.

Mejoras del sistema

- Implementación de un parser que permita identificar errores de sintaxis en el DSL cambiando el flujo: Lexer - Parser - Interpreter
- Panel de control web con visualización gráfica en tiempo real.
- Estrategias avanzadas de gestión energética mediante deep sleep y control dinámico de periféricos.
- Mejora de la experiencia de usuario en el dashboard.

7.6. Future Work

Although the developed system meets the main objectives of the project, several future improvement directions have been identified:

DSL extensions

- Event-driven execution mechanisms to reduce CPU usage.
- User-defined subroutines for reusable code blocks.
- Support for additional data types (strings, arrays, and simple structures).
- Direct access to ESP32-S3 internal peripherals (ADC, internal sensors).
- SPI communication support in addition to I2C.

Hardware evolution

- Design and implementation of a custom PCB replacing the current breadboard prototype.
- Integration of the system into a compact and portable form factor.
- Full validation of power management and thermal design.

System improvements

- Implementation of a parser capable of identifying syntax errors in the DSL by changing the flow: Lexer – Parser – Interpreter
- Advanced web dashboard with real-time graphical data visualization.
- Enhanced energy optimisation strategies using deep sleep and dynamic peripheral control.
- Improved user experience in the control interface.

7.7. Uso de la Inteligencia Artificial

La inteligencia artificial ha sido utilizada en este proyecto como una herramienta de apoyo al desarrollo, orientada a facilitar y agilizar el proceso de implementación. No obstante, el diseño del sistema, la arquitectura general y la toma de decisiones técnicas han sido responsabilidad exclusiva del autor, siendo la IA un recurso auxiliar y no determinante.

En las fases iniciales del proyecto, se utilizó NotebookLM con el objetivo de consultar y sintetizar documentación técnica extensa relativa a los distintos componentes hardware y software considerados. En algunos casos, dicha documentación superaba ampliamente las 300 páginas, por lo que esta herramienta permitió agilizar la comprensión y selección de tecnologías relevantes.

Posteriormente, se hizo uso de herramientas de IA generativa durante las primeras etapas de aprendizaje del entorno ESP-IDF, debido a la ausencia de sistemas de autocompletado avanzados (IntelliSense) en dicho entorno. Dado que el punto de partida del proyecto implicaba un conocimiento limitado del ecosistema ESP32, la IA se empleó como apoyo para comprender el funcionamiento del microcontrolador y sus interfaces de programación.

Asimismo, se utilizó como soporte para resolver dudas puntuales relacionadas con cabeceras, funciones específicas del ESP32 y diferencias entre versiones de la documentación oficial, las cuales en algunos casos resultaban inconsistentes o incompletas.

También se empleó para agilizar el desarrollo del interfaz web embebido, ya que el HTML se encontraba integrado dentro de cadenas en C++. Esto implicaba largos tiempos de compilación para cada iteración, por lo que la IA contribuyó a reducir el tiempo de desarrollo en esta fase.

De forma adicional, se utilizó como apoyo para la interpretación de mensajes de error y logs de depuración generados por el sistema, facilitando la identificación de problemas durante la fase de desarrollo.

En el caso del intérprete del DSL, la IA se utilizó únicamente como apoyo puntual para la comprensión de instrucciones complejas, como READLE, sin intervenir en el diseño del lenguaje ni en la implementación de su arquitectura.

Finalmente, se deja constancia de que la arquitectura general del sistema, el diseño del DSL y el contenido de esta memoria han sido desarrollados íntegramente por el autor, sin generación automática de contenido estructural o arquitectónico por parte de sistemas de inteligencia artificial.

7.8. Use of Artificial Intelligence

Artificial intelligence has been used in this project as a development support tool aimed at facilitating and accelerating the implementation process. However, the system design, overall architecture, and technical decision-making were exclusively carried out by the author, with AI serving only as an auxiliary resource.

In the early stages of the project, NotebookLM was used to consult and summarise extensive technical documentation related to the hardware and software components considered. In some cases, this documentation exceeded 300 pages, and the tool helped streamline the analysis and selection of relevant technologies.

Later, generative AI tools were used during the initial learning phase of the ESP-IDF environment, mainly due to the lack of advanced autocompletion (IntelliSense) support. Since the starting point of the project involved limited knowledge of the ESP32 ecosystem, AI was used to support the understanding of the microcontroller and its programming interfaces.

It was also used to resolve specific doubts regarding headers, ESP32-specific functions, and inconsistencies between different versions of the official documentation, which in some cases were incomplete or contradictory.

Additionally, AI was used to speed up the development of the embedded web interface, as the HTML code was embedded within C++ strings. This required recompilation after each modification, leading to long iteration cycles that were mitigated with the help of AI-generated suggestions.

Furthermore, it was used to assist in interpreting error messages and debugging logs generated by the ESP32, improving the efficiency of the debugging process.

Regarding the DSL interpreter, AI was only used for occasional clarification of complex instructions such as READLE, without contributing to the design or implementation of the language itself.

Finally, it is stated that the overall system architecture, DSL design, and the content of this document were entirely developed by the author, without the use of AI for generating structural or architectural components.

Capítulo 8

Presupuesto

El presente presupuesto está centrado en la estimación del coste asociado a la adquisición de los componentes utilizados a lo largo de este Trabajo de Fin de Grado, así como en la valoración del esfuerzo de desarrollo del sistema.

El mayor coste del proyecto se corresponde con la mano de obra, la cual engloba el diseño, implementación, integración y validación de los distintos módulos hardware y software, así como el consumo energético asociado durante las fases de desarrollo y pruebas.

8.1. Coste de mano de componentes

Tabla 8.1: Coste de los componentes principales del sistema

Descripción	Cantidad	Coste (€)
Placa ESP32 con WiFi + Bluetooth	1	17,80
Expansor de pines digitales (I2C)	1	4,25
Pantalla TFT color SPI 2" (176x220)	1	19,95
Módulo giroscopio y acelerómetro (IMU)	1	6,95
Sensor de temperatura	1	3,00
Kit de componentes electrónicos	1	6,90
Subtotal componentes		58,85 €

8.2. Coste de mano de obra

Tabla 8.2: Estimación del coste de desarrollo

Descripción	Valor
Precio por hora	40,00 €
Horas de trabajo estimadas	300 h
Subtotal mano de obra	12.000,00 €

8.3. Coste total del proyecto

Tabla 8.3: Estimación del coste de desarrollo

Concepto	Coste (€)
Componentes	58,85 €
Mano de obra	12.000,00 €
Coste total estimado del proyecto	12.058,85€

Capítulo 9

Bibliografía

- **ESP-IDF Programming Guide:** <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>
- **API Reference de redes (WiFi/HTTP server):** <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/index.html>
- **FreeRTOS en ESP-IDF:** <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos.html>
- **Partition tables:** <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/partition-tables.html>
- **Build system (CMake/components):** <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/build-system.html>
- **NVS / SPIFFS / FAT (almacenamiento + SD SPI):** <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/index.html>
- **Foro oficial Espressif:** <https://esp32.com/>
- **LovyanGFX (ya vendorizado):** <https://github.com/lovyan03/LovyanGFX>
- **I2C / patrón de bus en ESP-IDF:** <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/i2c.html>
- **cppreference:** <https://en.cppreference.com/>
- **Patrones de diseño (Observer / Service Locator):** <https://refactoring.guru/design-patterns>
- **ESP HTTP Server:** https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/protocols/esp_http_server.html
- **GitHub Actions ESP-IDF CI:** <https://github.com/espressif/esp-idf-ci-action>
- **MkDocs:** <https://www.mkdocs.org/>
- **Material for MkDocs:** <https://squidfunk.github.io/mkdocs-material/>

Apéndice A

Componentes del bloque Components

En este apéndice se describen los distintos componentes que forman parte del bloque components, encargados de proporcionar funcionalidades reutilizables dentro del sistema.

A.1. IO

El componente IO se encarga de abstraer el acceso a los distintos elementos de hardware del sistema que pueden ser utilizados tanto por el usuario como internamente por el propio dispositivo, ya sea mediante el DSL o a nivel de software.

Este módulo define una clase base que actúa como interfaz común, a partir de la cual derivan los siguientes componentes:

- Button: gestiona la interacción con los botones físicos del dispositivo, incluyendo la detección de pulsaciones y eventos asociados.
- Led: permite el control de los LEDs físicos, proporcionando una interfaz unificada para su activación, desactivación y configuración.
- Buzzer: gestiona el control de zumbadores, permitiendo generar señales acústicas a partir de eventos del sistema.

Esta abstracción facilita la reutilización del código, simplifica la integración de nuevos elementos de entrada/salida y mejora la mantenibilidad del sistema.

A.2. Lexer

El componente Lexer es el encargado de analizar las instrucciones escritas en el Domain Specific Language (DSL) del sistema.

Su función principal consiste en tokenizar las instrucciones de entrada, validando su sintaxis y generando una secuencia de tokens estructurados que posteriormente serán procesados por el intérprete.

Este proceso permite transformar el lenguaje definido por el usuario en una representación intermedia comprensible por el sistema.

A.3. Locator

El componente Locator implementa un patrón de localización de servicios (Service Locator), permitiendo el acceso centralizado a los distintos servicios registrados en el sistema.

A través de este mecanismo, es posible acceder a instancias de clases sin necesidad de gestionar referencias globales ni declarar dependencias estáticas, facilitando así la modularidad del sistema.

Este enfoque permite desacoplar los componentes del sistema y simplificar la gestión de dependencias dentro de la arquitectura.

A.4. Logger

El componente Logger unifica el sistema de registro de eventos del dispositivo.

Su objetivo es proporcionar una interfaz centralizada para la generación, gestión y personalización de logs, permitiendo clasificar la información en función del contexto o del subsistema que la genere.

Esto facilita tanto la depuración como el análisis del comportamiento del sistema durante su ejecución.

A.5. Observer

El componente Observer implementa el patrón de diseño Observer, permitiendo establecer un sistema de suscripción entre objetos.

A través de este mecanismo, los distintos componentes pueden registrar observadores sobre variables o eventos específicos, de forma que estos sean notificados automáticamente cuando se produzca un cambio de estado.

La respuesta a estos eventos se implementa mediante funciones lambda, lo que permite definir comportamientos dinámicos de forma sencilla.

Este enfoque evita la necesidad de realizar consultas continuas sobre el estado de las variables (polling), reduciendo la carga de procesamiento y mejorando la eficiencia del sistema.

Apéndice B

Especificación del lenguaje DSL

En este apéndice se describe el funcionamiento del lenguaje específico de dominio (Domain Specific Language, DSL) desarrollado para el sistema. Este lenguaje permite al usuario definir programas de forma simplificada, abstrayéndose de la complejidad del hardware subyacente y de la implementación interna en C++.

El DSL está diseñado para ejecutarse dentro del intérprete del sistema, el cual se apoya en componentes como el lexer para la tokenización y validación sintáctica, y el interpreter para la traducción de instrucciones a operaciones internas.

B.0.1. Estructura general del lenguaje

Cada archivo con extensión `.str` representa un programa independiente.

El lenguaje presenta las siguientes características generales:

- Cada línea contiene una única instrucción.
- Las líneas en blanco son ignoradas.
- Las palabras clave no son sensibles a mayúsculas/minúsculas.
- Los valores numéricos pueden expresarse en formato decimal o hexadecimal.
- Las cadenas de texto se delimitan mediante comillas dobles.
- La ejecución del programa es secuencial, comenzando desde la primera línea.

B.0.2. Ejemplo de inicio rápido

El siguiente ejemplo muestra un programa básico que declara un dispositivo LED y lo hace parpadear:

```
device = led name = myled pin = 17
```

```
loop 3
  write = myled on
  wait 1
  write = myled off
  wait 1
dloop
```

Este programa configura un LED en el pin 17 y lo activa de forma intermitente tres veces.

B.0.3. Definición de dispositivos

El lenguaje permite la declaración de dispositivos hardware mediante una sintaxis unificada:

```
device = <tipo> name = <identificador> pin = <número>
```

Ejemplos:

```
device = led name = myled pin = 17  
device = buzzer name = horn pin = 26  
device = button name = btn pin = 32
```

Tipos soportados

Tipo	Descripción
led	Salida digital. Permite escritura y lectura de estado.
buzzer	Salida digital de tipo acústica.
button	Entrada digital. Devuelve 1 si está pulsado y 0 en caso contrario.

B.0.4. Escritura en dispositivos

La escritura en dispositivos de salida se realiza mediante:

```
write = <nombre> on|off
```

Ejemplos:

```
write = myled on  
write = horn off
```

Los dispositivos de entrada, como los botones, no admiten escritura, ya que su estado es leído directamente durante la evaluación de expresiones.

B.0.5. Variables y operaciones

El lenguaje soporta variables numéricas y expresiones matemáticas:

```
counter = 5
total = counter + 3
result = (a + b) * 2
```

También se permite asignación directa de valores:

```
0 -> counter
0x76 -> sensor_addr
```

Operadores soportados

- Aritméticos: + - * / %
- Bit a bit: & |
- Desplazamiento: « »
- Agrupación: ()

La evaluación se realiza de izquierda a derecha, recomendándose el uso de paréntesis para controlar la precedencia.

B.0.6. Control de flujo

Condicionales

```
if counter > 5
    write = myled off
else
    write = myled on
endif
```

Operadores soportados:

```
== != < <= > >=
```

Las condiciones pueden evaluarse sobre variables, dispositivos o entradas digitales.

Bucles

El lenguaje permite tres tipos de bucles:

```
loop 5
```

```
...
```

```
dloop
```

```
loop -1
```

```
...
```

```
dloop
```

```
loop counter < 10
```

```
  counter = counter + 1
```

```
dloop
```

El intérprete introduce pequeñas pausas entre iteraciones para evitar bloqueos del sistema.

B.0.7. Sistema de logs e impresión

El sistema permite redirigir la salida a archivos de log:

```
file "sesion.log"
```

Impresión de datos:

```
print "Lectura:" sensor "°C"
```

```
print "Estado=" counter
```

Cada mensaje incluye información temporal y se almacena en el archivo activo.

B.0.8. Comunicación I2C

El DSL incorpora primitivas para comunicación con dispositivos I2C.

Inicialización:

```
i2c init
```

Escritura:

```
i2c write 0x76 0xF4
```

```
i2c write 0x76 0xF4 0x25
```

Lectura:

```
i2c read 0x76 0xFA 3 -> temp_raw  
i2c readle 0x76 0x88 2 -> cal_T1
```

El sistema soporta formatos big-endian y little-endian, con conversión automática a enteros con signo cuando es necesario.

B.0.9. Expansor de pines (PCF8574)

El sistema integra soporte para el expansor PCF8574, accesible mediante I2C, permitiendo ampliar el número de entradas y salidas digitales.

Se pueden definir alias para los pines:

```
expin myLed = 0  
expin myBut = 1
```

Escritura:

```
i2c write pin=myLed HIGH  
i2c write pin=3 LOW
```

Lectura:

```
i2c read pin=myBut -> estado
```

El sistema mantiene una copia interna del estado de los pines para evitar conflictos en la comunicación.

B.0.10. Utilidades

Conversión a entero con signo de 16 bits:

```
sign16 raw
```

B.0.11. Ejemplo completo

```
file "blink.log"
```

```
device = led name = led1 pin = 17  
device = button name = btn pin = 32  
count = 0
```

```
loop -1  
  if btn == 1  
    write = led1 on  
    count = count + 1  
    print "Pulsado n: " count  
    wait 0.5  
  else  
    write = led1 off  
  endif  
  wait 0.05  
dloop
```

B.0.12. Referencia rápida

Categoría	Elementos
Dispositivos	device, led, button, buzzer, name, pin, write
Control	if, else, endif, loop, dloop
Variables	=, ->
Salida	print, file
Tiempo	wait
Comparación	==, !=, <, <=, >, >=
Operadores	+, -, *, /, %, &, , «, »
I2C	i2c init, write, read, readle
Expansión	expin, high, low
Utilidades	sign16